Операционные системы

Тема: Понятие процесса, виды процессов и его состояния в операционных системах

Информация о докладчике и преподавателе

Докладчик

- Юсупова Амина Руслановна
- студентка НКАбд-06-25
- Российский университет дружбы народов им. Патриса Лумумбы
- 1032253525@rudn.ru

Преподаватель

- Дмитрий Сергеевич Кулябов
- д.ф.-м.н., профессор
- профессор кафедры теории вероятностей и кибербезопасности

Цели и задачи работы

Цель работы

Целью данной работы является объяснение понятия процесса как одно из ключевых понятий операционных систем

Задачи:

1. Дать определение процессу и объяснить, чем он отличается от программы.
2. Перечислить и кратко охарактеризовать основные виды процессов
3. Описать состояния процесса и их смену.

Гипотеза и ход работы

Гипотеза

- Вероятно, глубокое понимание концепции процесса, его классификации и особенно модели состояний является необходимым условием для понимания принципов функционирования операционной системы в целом.

Ход работы:

- 1. Поиск и анализ теоретического материала
- 2. Структурирование информации и выделение ключевых разделов
- 3. Углубленное изучение темы программных потоков
- 4. Подготовка текста доклада и презентации

Что такое процесс?

Процесс — это фундаментальное понятие ОС, включающее два аспекта: носитель данных (владеет ресурсами и памятью),исполнитель операций (выполняет код).

- Главное отличие от программы:

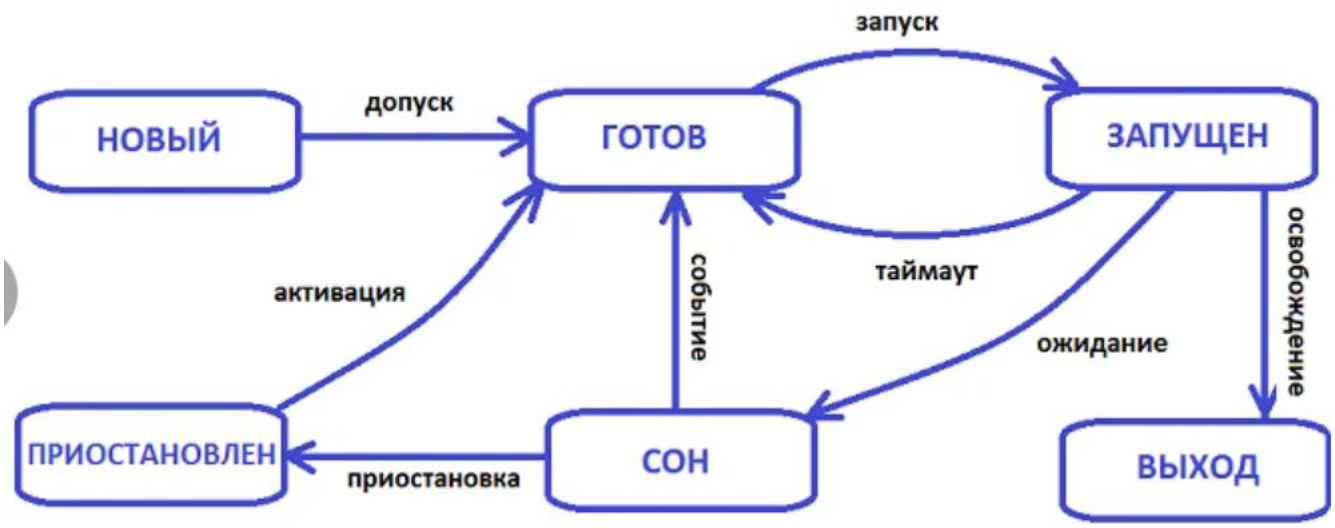
Программа — пассивный объект (файл на диске).

Процесс — активный объект (выполняющийся экземпляр программы).

Состояния процесса

Процесс в ходе исполнения проходит несколько состояний

- Порождение: Подготовка к первому запуску.
- Активное: Программа напрямую исполняется процессором.
- Ожидание: Процесс остановлен, так как ждет ресурс
- Готовность: У процесса есть все ресурсы, кроме одного — самого процессора
- Окончание: Завершение работы (нормальное или аварийное).



Роль ОС в управлении процессами

Задачи операционной системы

Для ОС процесс — это объект управления.

Главная задача ОС:

- Обеспечить нахождение процесса в каждом из состояний.
- Контролировать правильные переходы между состояниями.

Причины переходов (события):

- Инициатива самого процесса (запрос ресурса).
- Внешние события (прерывания от устройств или таймера).

Классификация процессов

По времени существования:

- Реального времени: Должны завершиться к строго определенному моменту.
- Интерактивные: Время реакции на пользователя должно быть минимальным.
- Пакетные: Обычные задачи, без жестких временных ограничений.

По происхождению:

- Порождающие: Создающие другие процессы.
- Порожденные: Созданные другими процессами (дочерние).

Классификация процессов

По принадлежности кода:

- Системные: Исполняют код ядра ОС.
- Пользовательские: Исполняют код прикладных программ.

По связи друг с другом:

- Взаимосвязанные: Обмениваются данными или ресурсами.
- Изолированные: Не имеют явных связей.
- Конкурирующие: Борются за одни и те же ресурсы (например, за принтер)

Правила сосуществования

- **Предшествование**: Процесс А должен строго выполняться раньше процесса Б.
- **Приоритетность**: Процесс с высоким приоритетом всегда выполняется раньше процесса с низким.
- **Взаимное исключение**: Два процесса не могут одновременно работать с одним ресурсом (например, записывать данные в один файл). Участки кода, где происходит работа с общим ресурсом, называются **критическими областями**

Программные потоки

Поток — это "облегченный" процесс, который работает внутри другого процесса.

- Отличие: Обычный процесс имеет свое отдельное адресное пространство. Потоки одного процесса делят память и ресурсы между собой.

Виды потоков

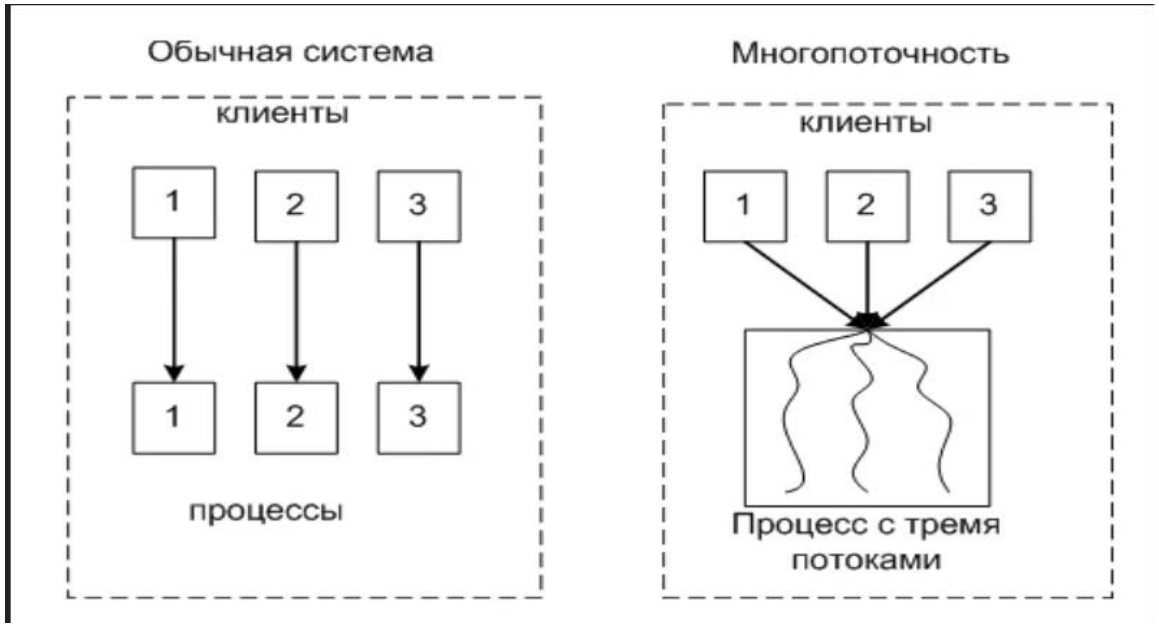
- **Однопоточность**: Программа делает только одно дело за раз.
- **Многопоточность**: Программа может делать несколько дел одновременно (например, браузер загружает страницу и одновременно крутит анимацию).

Преимущества многопоточности и итоги

Зачем нужны потоки ?

Преимущество многопоточности:

- Скорость
- Экономия памяти
- Удобство



Заключение

1. Установлено ключевое различие: программа — это пассивный код на диске, а процесс — активный объект, потребляющий ресурсы. Именно это превращает статичные файлы в реальную работу компьютера.

2. Деление процессов на виды (системные/пользовательские, пакетные/интерактивные) и отношения между ними (приоритетность, взаимное исключение) демонстрируют гибкость ОС. Это позволяет ей эффективно управлять задачами от фоновых до критичных ко времени.

3. Модель состояний (Создание → Готовность → Исполнение → Ожидание → Завершение) объясняет механизм многозадачности. Введение потоков (нитей) показывает эволюцию идеи: "облегченные" процессы внутри одной программы ускоряют работу и экономят ресурсы.